

Decision Twin

An Agentic AI Operating System for Manufacturing Decisions

Executive Summary

Decision Twin is an MCP-native, multi-agent system that models how a manufacturing plant makes operational decisions. Rather than one general-purpose assistant, a coordinated team of specialized agents plans, gathers evidence through tools, retrieves institutional memory, challenges its own conclusions, simulates alternative outcomes, and converges on a single executable action plan. Manufacturing is the proving ground; the core contribution is a demonstration of autonomous planning, tool-use, memory, reflection, and multi-agent coordination working together as one system.

Problem Statement

Factories generate continuous streams of sensor, maintenance, production, inventory, and quality data. AI can already detect anomalies well. The unsolved problem is what happens next: deciding what to do about an anomaly requires correlating disconnected information across departments that often have conflicting priorities — production wants throughput, safety wants caution, finance wants cost control. Today this correlation is done manually and slowly, leading to avoidable downtime, inconsistent decisions, and no auditable reasoning trail. The gap is not detection. It is autonomous, explainable decision-making.

Vision

Digital Twins simulate machines. Decision Twin simulates the organization's decision-making process itself — a live, reasoning model of how a plant should think, staffed by autonomous agents instead of people in a meeting room.

Core Agentic Capabilities Demonstrated

The architecture is organized around five capabilities that define agentic AI systems, not around departments. Manufacturing roles are assigned within this structure, not the other way around.

Capability	What It Looks Like In This System
Planning	Planner Agent decomposes an incoming event into sub-goals dynamically; no fixed if-else workflow.
Tool-Use	Each agent decides which MCP tool to call and when, based on evidence already gathered, not a fixed script.
Memory	Historical Memory Agent is queried mid-reasoning by other agents, not just once at the start.
Reflection	Devil's Advocate Agent critiques the leading proposal and can force a re-plan; a genuine self-critique loop.
Coordination	Agents negotiate over a shared state/blackboard and must reach explicit consensus, with dissent logged.

System Workflow

- 1 Sensor anomaly or operational event detected.
- 2 Planner Agent decomposes the event into a sub-goal list (what needs investigating, not a fixed checklist).
- 3 Plant Manager Agent activates only the agents relevant to that goal list.
- 4 Tool-use agents gather evidence autonomously via MCP tools, deciding what to call and when.
- 5 Historical Memory Agent is queried inline by any agent that needs precedent.
- 6 Agents negotiate over shared state; conflicting objectives are surfaced explicitly.
- 7 Devil's Advocate Agent challenges the leading proposal; may trigger another negotiation round.
- 8 Scenario Simulation Agent evaluates 2-3 alternative actions by re-invoking agents against hypothetical state.
- 9 Decision Engine (folded into Plant Manager) converges on one plan, with confidence and dissent recorded.
- 10 Executable outputs generated: work order, technician assignment, schedule update, supervisor notification.

Agent Architecture

Twelve agents organized into five functional layers. Each agent has a distinct reasoning responsibility; none exist merely to inflate agent count.

1. Planning Layer

Agent	Responsibility
Planner Agent	Decomposes the incoming event into a dynamic sub-goal list. Core planning behaviour: decides what needs
Plant Manager Agent	Meta-controller. Spawns and retires agents based on the Planner's goal list, tracks task-graph state, and fold

2. Tool-Use Layer

Agent	Responsibility
Sensor Agent	Reads live sensor data via <code>get_sensor_data()</code> , flags anomalies against dynamic thresholds.
Maintenance Agent	Calls <code>check_machine_health()</code> ; decides whether more evidence is needed before concluding severity.
Inventory Agent	Calls <code>check_inventory()</code> ; only activated if Maintenance Agent flags a probable part failure.
Finance Agent	Calls <code>estimate_downtime_cost()</code> ; triggered conditionally, e.g. only once Maintenance confirms high severity.

3. Memory Layer

Agent	Responsibility
Historical Memory Agent	RAG over incident reports and past failures (FAISS/ChromaDB). Queried mid-reasoning by other agents, not

4. Reflection & Safety Layer

Agent	Responsibility
Devil's Advocate Agent	Actively challenges the leading proposal, requests missing evidence, and can force a renegotiation round. Im
Safety Agent	Validates SOP compliance; holds veto power over any plan that violates a safety rule, not merely advisory in
Risk Agent	Computes an operational risk score combining safety, financial, and schedule inputs.

5. Simulation & Synthesis Layer

Agent	Responsibility
Quality Agent	Assesses downstream quality implications of each candidate action.
Scenario Simulation Agent	Runs counterfactual comparisons (repair-now / delay-repair / reduced-capacity) by re-invoking relevant agent
Production Agent	Evaluates schedule and delivery-commitment impact for each simulated option, closing the loop back to the P

MCP Resources

- Sensor streams
- Maintenance history
- Production schedule
- Inventory database
- SOP manuals
- Incident reports
- Supplier database
- Quality reports
- Technician roster

Representative MCP Tools

- `get_sensor_data()` — live sensor stream access
- `check_machine_health()` — maintenance-history-derived health score
- `search_incident_history()` — RAG retrieval over past incidents
- `check_inventory()` — spare-parts availability
- `estimate_downtime_cost()` — financial impact projection
- `calculate_risk()` — composite operational risk score
- `simulate_scenario()` — counterfactual outcome projection
- `generate_work_order()` — executable output
- `notify_supervisor()` — alerting
- `update_schedule()` — production reroute

Technology Stack

Layer	Choice
Agent Orchestration	LangGraph — conditional edges model negotiation rounds and reflection loops
MCP Layer	NitroStack MCP Server
Backend	FastAPI
Knowledge Layer	RAG + FAISS / ChromaDB
Frontend	React + Tailwind CSS — live agent transcript + LangGraph state visualization
LLM	GPT-5.5 / Llama / Gemma (hackathon-provided access)

Demo Scenario

Machine #4 reports abnormal vibration and rising temperature. The Planner Agent decomposes the event into sub-goals (assess health, check precedent, evaluate schedule impact, check parts). Plant Manager activates Sensor, Maintenance, Historical Memory, Production, Inventory, and Finance agents. Maintenance flags moderate-to-high severity; Historical Memory surfaces a near-identical past incident where a delayed repair led to a larger failure. This precedent shifts the group's leading proposal toward immediate repair. Devil's Advocate challenges the proposal, asking whether current load actually matches the historical case, forcing Sensor and Production agents to re-verify. Scenario Simulation Agent then compares three options — repair-now, delay-repair, and reduced-capacity — each simulated by re-invoking Finance, Safety, and Production against hypothetical states. Safety Agent vetoes the delay option outright due to SOP violation risk at current vibration levels. Plant Manager converges on repair-now, generates a work order, reserves the required spare part, assigns an available technician, reroutes production, and notifies the supervisor — with the full negotiation transcript, including the Devil's Advocate challenge and Safety veto, shown as the explanation.

Innovation / Differentiation

- Decision Twin instead of a conventional Digital Twin — models organizational reasoning, not machine state.
- Agents are organized by agentic capability (planning, tool-use, memory, reflection, coordination), not by department — the structure itself is the innovation, manufacturing is the application.
- Genuine reflection loop: Devil's Advocate can force re-negotiation, not just append a comment.
- Memory queried mid-reasoning, not only as a preprocessing step, so precedent can change an in-progress conclusion.
- Counterfactual scenario simulation by re-invoking real agents against hypothetical state, not a separate static model.
- Executable outputs (work order, technician assignment, schedule update) rather than a recommendation a human must still act on.
- Full negotiation transcript, including dissent and vetoes, shown as the explanation — explainability by construction, not a bolt-on summary.

Expected Impact

Faster response to operational anomalies, transparent and auditable cross-department reasoning, fewer avoidable downtime incidents, and a concrete demonstration that autonomous multi-agent systems can produce trustworthy, explainable, executable decisions — not just insights.

24-Hour Build Roadmap

Phase	Scope
Hours 0-4	Plant Manager + Planner dynamic activation logic; MCP server with 4-5 core tools stubbed.
Hours 4-9	Sensor, Maintenance, Finance agents wired to real/mock MCP tools with conditional tool-call logic.
Hours 9-13	Devil's Advocate reflection loop (minimum 2 negotiation rounds) + Historical Memory RAG integration.
Hours 13-17	Scenario Simulation Agent (numeric/rule-based, not heavy ML) + Safety veto logic.
Hours 17-21	Consensus synthesis into Plant Manager; executable outputs (work order, notification, schedule update).
Hours 21-24	Frontend transcript view + LangGraph state visualization; demo rehearsal on Machine #4 scenario.

Priority if time runs short: a working 6-agent core (Planner, Plant Manager, 2 tool-use agents, Devil's Advocate, Safety) with a genuine 2-round reflection loop outperforms a shallow 12-agent system where every agent is a one-shot call.